

DAY 4 BACKEND TASKS

Nandini Bhimineni

Yoganandini9@gmail.com

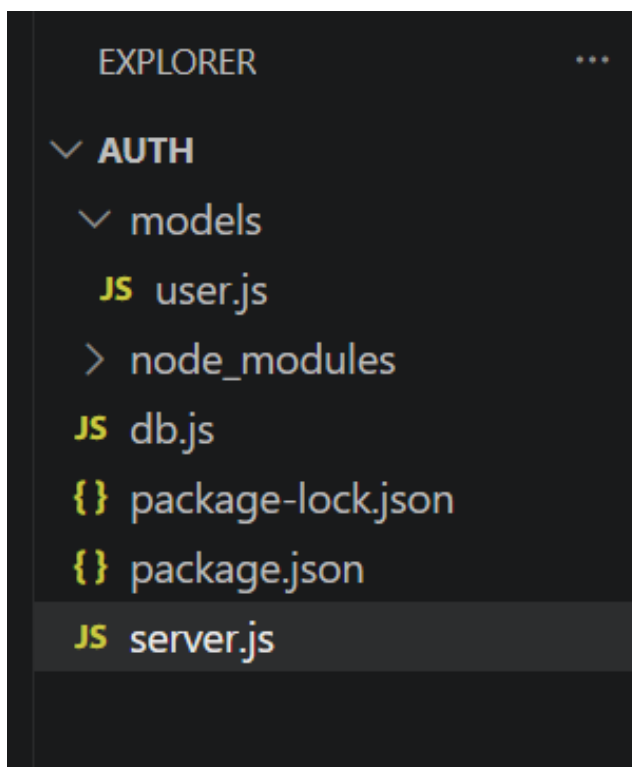
Backend developer Intern

Task 1 — User Registration API

1. Create a POST /register API.
2. Accept the following fields:
 - name
 - email
 - password
3. Store user data in MongoDB.
4. Hash the password using bcrypt before storing.
5. Return a success message after registration.

VS CODE:

Structure



A screenshot of a VS Code editor window with three tabs: 'Welcome', 'JS server.js', and 'JS db.js'. The 'server.js' tab is active, showing a JavaScript file with 44 lines of code. The code sets up an Express.js server with a Home API endpoint and a Register API endpoint. The Register API endpoint includes validation for empty fields, password length, and duplicate email checks. The code uses ES6 features like const, async/await, and arrow functions. The editor has a dark theme and a sidebar on the left showing the file explorer.

```
JS server.js > ...
1  const express = require("express");
2  const bcryptjs = require("bcryptjs");
3
4  require("./db");
5
6  const User = require("./models/User");
7
8  const app = express();
9
10 app.use(express.json());
11
12
13 // Home API
14 app.get("/", (req, res) => {
15   res.send("API is working");
16 });
17
18
19 // Register API
20 app.post("/register", async (req, res) => {
21
22   const { name, email, password } = req.body;
23
24   // Empty fields validation
25   if (!name || !email || !password) {
26     return res.json({
27       message: "All fields are required"
28     });
29   }
30
31   // Password length validation
32   if (password.length < 6) {
33     return res.json({
34       message: "Password must be at least 6 characters"
35     });
36   }
37
38   // Duplicate email check
39   const existingUser = await User.findOne({ email });
40
41   if (existingUser) {
42     return res.json({
43       message: "Email already registered"
44     });
45   }
46 }
```

```

45     }
46
47     // Hash password
48     const hashedPassword = await bcryptjs.hash(password, 10);
49
50     // Save user
51     const user = new User({
52       name,
53       email,
54       password: hashedPassword
55     });
56
57     await user.save();
58
59     res.json({
60       message: "User registered successfully"
61     });
62   });
63
64
65
66   // Login API
67   app.post("/login", async (req, res) => {
68
69     const { email, password } = req.body;
70
71     // Check user
72     const user = await User.findOne({ email });
73
74     if (!user) {
75       return res.json({
76         message: "User not found"
77       });
78     }
79
80     // Compare password
81     const isMatch = await bcryptjs.compare(
82       password,
83       user.password
84     );
85
86     if (!isMatch) {
87       return res.json({

```

```

88         message: "Invalid Password"
89       });
90     }
91
92     res.json({
93       message: "Login successful"
94     });
95   });
96
97
98
99   // Get All Users API
100  app.get("/users", async (req, res) => {
101
102    const users = await User.find();
103
104    res.json(users);
105  });
106
107
108
109  // Delete User API
110  app.delete("/user/:id", async (req, res) => {
111
112    await User.findByIdAndDelete(req.params.id);
113
114    res.json({
115      message: "User deleted successfully"
116    });
117  });
118
119
120
121  // Server
122  app.listen(5000, () => {
123    console.log("Server is running on port 5000");
124  });

```

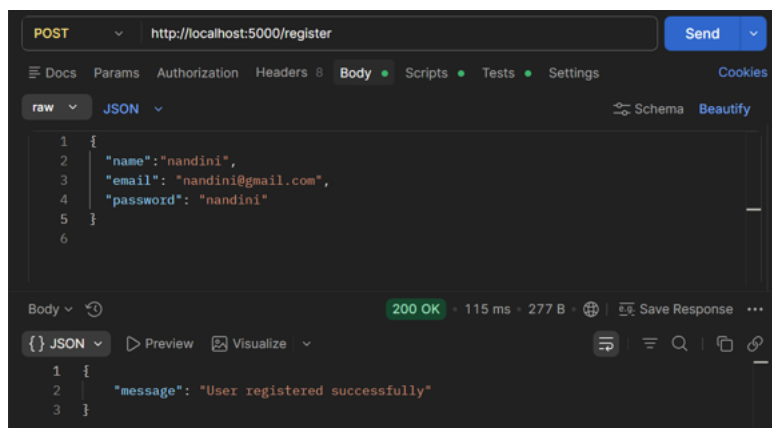
Db.js

```
JS db.js > ...
1  const mongoose = require("mongoose");
2
3  mongoose.connect("mongodb://localhost:27017/authdb")
4  .then(() => console.log("MongoDB Connected"))
5  .catch(err => console.log(err));
```

User.js

```
models > JS user.js > ...
1  const mongoose = require("mongoose");
2
3  const userSchema = new mongoose.Schema({
4    name: String,
5    email: String,
6    password: String
7  });
8
9  module.exports = mongoose.model("User", userSchema);
```

POST/registerAPI



DID other users registrations too naming : nandu, nandini, Priyanka, hrishi teja, kireety, Yashawini, Akshith

Along with their Emails and Password And connected to Mongo DB

Task 2 — User Login API

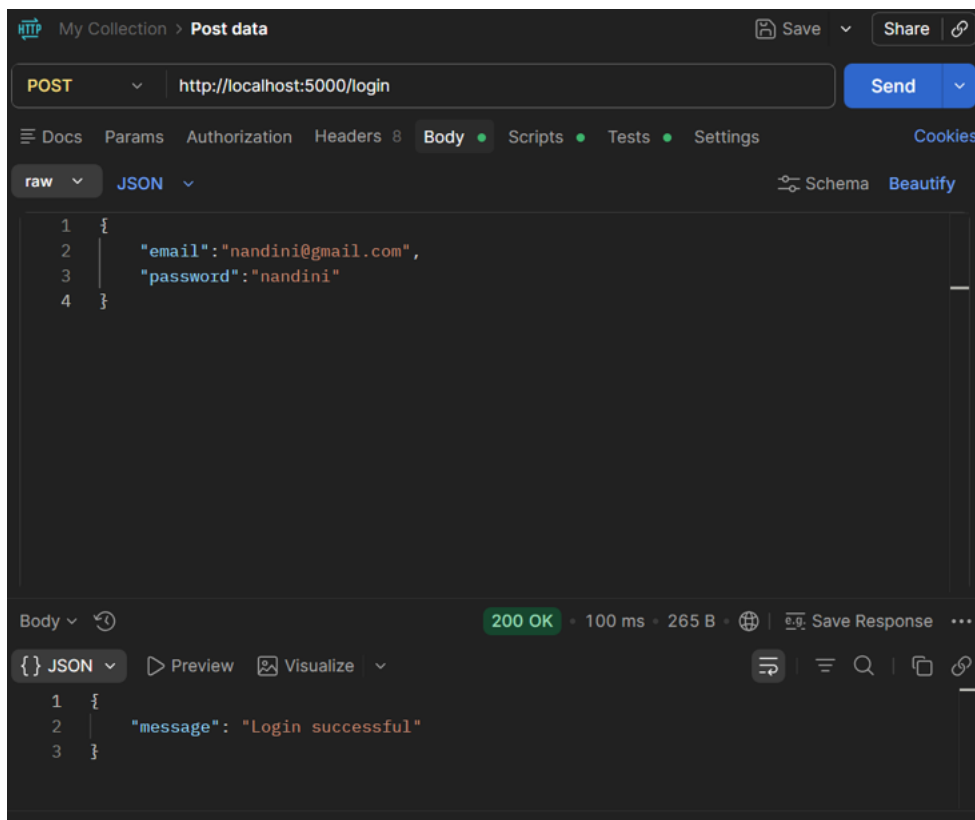
1. Create a POST /login API.

2. Accept:

- email
- password

5. Return:

- Login Successful
- Invalid Password
- User Not Found



Checking whether the user exists in MongoDB.

```
// Check user
const user = await User.findOne({ email });

if (!user) {
  return res.json({
    message: "User not found"
  });
}
```

User.findOne() searches MongoDB

It checks for matching email

If user exists → returns user data

If not → returns null

4. Compare the entered password with the hashed password.

When a user registers, the password is encrypted (hashed) using

bcryptjs before storing in MongoDB for security.

During login, the entered password is compared with the stored hashed password using:

bcryptjs.compare()

If both passwords match:

→ Login Successful

If passwords do not match:

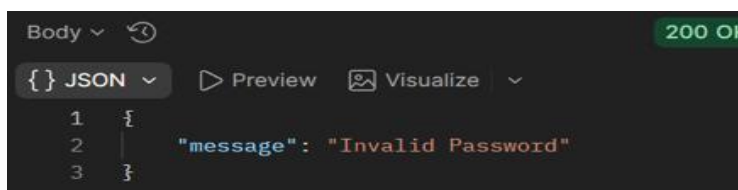
→ Invalid Password

```
// Compare password
const isMatch = await bcryptjs.compare(
  password,
  user.password
);

if (!isMatch) {
  return res.json({
    message: "Invalid Password"
  });
}

res.json({
  message: "Login successful"
});
});
```

This helps keep user passwords secure and prevents storing plain text passwords in the database



Task 3 — User Management APIs

1. Create the following APIs:

- GET /users
- DELETE /user/:id

2. Fetch all registered users from MongoDB.

3. Delete a user using their ID.

4. Test APIs using Postman.

PUT Get data POST Get data GET Post data + No environment Save Share

My Collection > Post data

GET http://localhost:5000/users Send

Docs Params Authorization Headers 6 Body Scripts Tests Settings Cookies

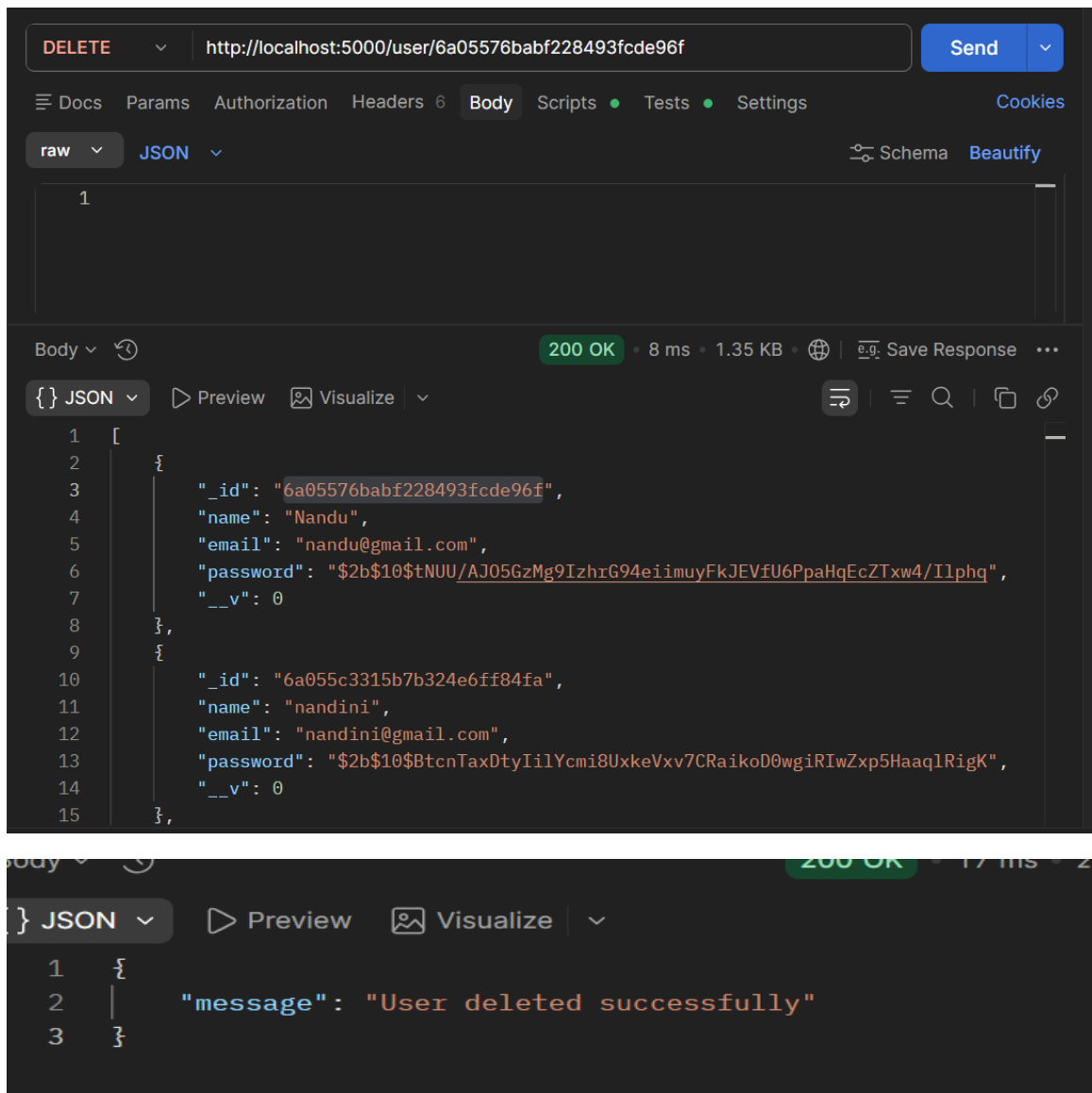
Body 200 OK 19 ms 1.35 KB Save Response

{ } JSON Preview Visualize

```
1  [
2      {
3          "_id": "6a05576babf228493fcde96f",
4          "name": "Nandu",
5          "email": "nandu@gmail.com",
6          "password": "$2b$10$tNUU/AJ05GzMg9IzhrG94eiimuyFkJEVfU6PpaHqEcZTxw4/I1phq",
7          "__v": 0
8      },
9      {
10         "_id": "6a055c3315b7b324e6ff84fa",
11         "name": "nandini",
12         "email": "nandini@gmail.com",
13         "password": "$2b$10$BtcnTaxDtyIilYcmi8UxkeVxv7CRaikoD0wgiRiWZxp5HaaqlRigK",
14         "__v": 0
15     },
16     {
17         "_id": "6a055c6215b7b324e6ff84fb",
18         "name": "priyanka",
19         "email": "priyanka@gmail.com",
20         "password": "$2b$10$X2nRvMGoI5W0hhM5SyN81epZCfP.yhi6kMH5huCkg9z9WmFn82c1m",
21         "__v": 0
22     },
```

```
23     {
24         "_id": "6a055c7b15b7b324e6ff84fc",
25         "name": "hrishi teja",
26         "email": "hrishiteja@gmail.com",
27         "password": "$2b$10$hTijxixRGxhyErBOKK5MK.L40mqmcLQ5LNVuW2FjTL9HNTfgraZke",
28         "__v": 0
29     },
30     {
31         "_id": "6a055c8e15b7b324e6ff84fd",
32         "name": "kireety",
33         "email": "kireety@gmail.com",
34         "password": "$2b$10$uX3B80db4TrDxLgis0PYJuyT9cZYDY2tbIZ2aVqlpqm2Lvej.AWe",
35         "__v": 0
36     },
37     {
38         "_id": "6a055ca515b7b324e6ff84fe",
39         "name": "yashaswini",
40         "email": "yashaswini@gmail.com",
41         "password": "$2b$10$u/YakKsf0qP8gb5HBEPaT0be5vvPNE0FSwgGqgGZAwV.NyKH/Xsm",
42         "__v": 0
43     },
44     {
```

```
44     {
45         "_id": "6a055cc315b7b324e6ff84ff",
46         "name": "akshith",
47         "email": "akshith@gmail.com",
48         "password": "$2b$10$1VtlKae42qYtJ.Atdmuyq0iRNjMw0/ofL4XIvPMtJ0qPVka6p.MC6",
49         "__v": 0
50     }
51 ]
```



1. In `server.js`, the GET `/users` API is implemented to retrieve all registered users stored in MongoDB.
2. When the GET `/users` API is called, the server uses `User.find()` to fetch all user records from the database.
3. The fetched user data is returned as a JSON response containing user details such as name, email, and password.
4. This API helps in viewing and managing all registered users in the authentication system.
5. In `server.js`, the DELETE `/user/:id` API is implemented to delete a user from MongoDB using the user's unique ID.
6. The user ID is passed as a parameter in the API URL using `req.params.id`.

7. The server uses `findByIdAndDelete()` to locate and remove the corresponding user record from MongoDB.

8. After successful deletion, the API returns the message “User deleted successfully”.

9. If an invalid user ID is provided, the deletion process will not remove any user from the database.

10. Both GET and DELETE APIs are tested using Postman to verify correct API functionality and database operations.

11. These APIs improve user management by allowing administrators to view and remove registered users from the system.

Task 4 — Validation & Security

1. Prevent duplicate email registrations.

2. Add validation for empty fields.

3. Ensure password contains minimum 6 characters.

4. Display proper error messages for invalid inputs.

1. In `server.js`, duplicate email registration prevention is implemented inside the `/register` API. Before saving a new user into MongoDB, the server checks whether the entered email already exists in the database using `User.findOne({ email })`. If the email is already registered, the API stops the registration process and returns the message “Email already registered”. This helps maintain unique user accounts and avoids duplicate registrations.

2. Empty field validation is implemented in the registration API to ensure that users provide all required details such as name, email, and password. The server checks whether any of these fields are missing or empty before processing the request. If any field is empty, the API returns the message “All fields are required”. This improves data accuracy and prevents incomplete user information from being stored in MongoDB.

3. Password validation is added in `server.js` to improve account security. Before storing user data in MongoDB, the server checks whether the entered password contains at least 6 characters. If the password length is less than 6 characters, the API returns the message “Password must be at least 6 characters”. This prevents users from creating weak passwords and improves authentication security.

4. Proper error handling and validation messages are implemented throughout the authentication system. During login, if the entered email does not exist in MongoDB, the API returns “User not found”. If the entered password does not match the hashed password stored in the database, the API returns “Invalid Password”. These error messages help users understand the exact issue and improve the usability of the authentication system.